

ЗВІТ

з лабораторної роботи №3 з дисципліни «Основи програмної інженерії»

1. Тема та мета лабораторної роботи

Тема: Модульне тестування програмного коду (Unit Testing).

Мета: Набуття практичних навичок із написання модульних тестів із використанням промислових фреймворків тестування. Оволодіння формальними техніками проєктування тестів — еквівалентне розбиття (Equivalence Partitioning) та аналіз граничних значень (Boundary Value Analysis). Отримання досвіду інтерпретації метрик покриття коду (code coverage) та ітеративного поліпшення тестового набору до досягнення порогу покриття рядків не менше 80%.

Посилання на ПЗ 01, ПЗ 02, ЛР 02:

ПЗ 01:

https://docs.google.com/document/d/1ag1iKpN7Jebp5KrYup1LotXpuAuHj_DO6RZ8wSSHJyE/edit?usp=sharing

ПЗ 02:

<https://nure-team-opi4-2.atlassian.net/jira/software/projects/CFO42/summary>

ЛР 02:

<https://github.com/artemrohozian/OPI-project-ContentFlow/blob/main/README.md>

2. Вихідний код реалізованого модуля з коментарями.

Administrator.js

```
const User = require('./User');

class Administrator extends User {

  constructor(adminId, email, passwordHash) {

    super(email, passwordHash);
```

```
        this.adminId = adminId;
    }

    // Метод для роботи з користувачами
    manageUsers(users) {

        console.log("Список користувачів:");

        // Виводимо пошту кожного користувача через цикл
        for (const user of users) {

            console.log(`- ${user.email}`);

        }

    }

}

module.exports = Administrator;
```

Client.js

```
const User = require('./User');

const WordPressSite = require('./WordPressSite');

class Client extends User {

    constructor(clientId, email, passwordHash, tariffPlan) {

        super(email, passwordHash);

    }

}
```

```
this.clientId = clientId;

this.tariffPlan = tariffPlan;

this.sites = [];

}

// Метод для підключення сайту

connectSite(url, apiKey) {

    try {

        // Перевіряємо, чи правильне посилання (умова)

        if (!url || !url.startsWith("https://")) {

            throw new Error("Некоректний URL сайту");

        }

        // Перевіряємо довжину ключа (умова)

        if (apiKey.length < 5) {

            throw new Error("API ключ занадто короткий");

        }

        // Створюємо сайт і додаємо його в масив

        const site = new WordPressSite(this.sites.length +

1, url);

        site.connect();

    }

}
```

```
        this.sites.push(site);

        console.log(`Сайт ${url} успішно підключено`);

        return true;

    } catch (error) {

        // Якщо щось пішло не так, ловимо помилку тут

        console.log("Помилка підключення:", error.message);

        return false;

    }

}

}

module.exports = Client;
```

Preset.js

```
class Preset {

    constructor(presetId, toneOfVoice, language) {

        this.presetId = presetId;

        this.toneOfVoice = toneOfVoice;

        this.language = language;

    }

}
```

```
// Додаємо налаштування мови та тону до тексту

applyToContent(content) {

    content.bodyText +=

        `\n\n[Мова: ${this.language}, Tone:
${this.toneOfVoice}]`;

}

}

module.exports = Preset;
```

SEOContent.js

```
class SEOContent {

    constructor(contentId, title) {

        this.contentId = contentId;

        this.title = title;

        this.bodyText = "";

    }

    // Метод для створення SEO-тексту

    generateContent(keywords) {

        // Якщо масив слів порожній, то нічого не робимо

        if (!Array.isArray(keywords) || keywords.length === 0)

        {

            console.log("Список ключових слів порожній");
```

```
        return;
    }

    this.bodyText = `SEO опис для товару
"${this.title}":\n`;

    // Проходимося по кожному слову через цикл

    for (const keyword of keywords) {

        // Додаємо слово, тільки якщо воно довше 2 символів

        if (keyword.length > 2) {

            this.bodyText += `- Оптимізований опис для
ключового слова "${keyword}":\n`;

        }

    }

    console.log("Контент успішно згенеровано");

}

}
```

module.exports = SEOContent;

User.js

```
class User {

    constructor(email, passwordHash) {
```

```
// Не даємо створити просто "Користувача" без ролі

if (new.target === User) {

    throw new Error("Неможливо створити об'єкт
абстрактного класу User");

}

this.email = email;

this.passwordHash = passwordHash;

}

// Метод для входу в систему

login(inputEmail, inputPassword) {

    // Перевіряємо, чи співпадають пошта і пароль

    if (

       inputEmail === this.email &&

        inputPassword === this.passwordHash

    ) {

        console.log("Успішний вхід у систему");

        return true;

    }

    console.log("Невірний email або пароль");
```

```
        return false;
    }

    logout() {

        console.log("Користувач вийшов із системи");

    }

}

module.exports = User;
```

WordPressSite.js

```
class WordPressSite {
    constructor(siteId, url) {
        this.siteId = siteId;
        this.url = url;
        this.isConnected = false;
    }

    connect() {
        this.isConnected = true;
    }

    // Метод для публікації контенту
    deployContent(content) {
        // Перевіряємо, чи взагалі підключений сайт
        if (!this.isConnected) {
            console.log("Сайт не підключений");
            return false;
        }
    }
}
```



```
// Перевіряємо, чи є текст для публікації
if (!content.bodyText) {
    console.log("Контент порожній");
    return false;
}

console.log("Контент успішно опубліковано:");
console.log(content.bodyText);
return true;
}

getStatus() {
    return this.isConnected ? "Підключено" : "Не
підключено";
}
}

module.exports = WordPressSite;
```

3. Таблиця проєктування тестів: «Тест-кейс | Вхідні дані | Очікуваний результат | Техніка (EP/BVA) | Статус (pass/fail)».

Таблиця проєктування тестів (EP та BVA)

Метод connectSite(url, apiKey)

№	Вхідні дані	Клас еквівалентності (EP)	Граничне значення (BVA)	Очікуваний результат
1	https://site.com, apiKey123	Коректні дані	API key > 5 символів	Сайт успішно підключено
2	"", apiKey123	Некоректний URL	Порожній рядок	Помилка підключення
3	http://site.com, apiKey123	Некоректний URL	Відсутній HTTPS	Помилка підключення
4	https://site.com, 1234	Некоректний API key	4 символи	Помилка підключення
5	https://site.com, 12345	Коректний API key	5 символів	Успішне підключення
6	null, apiKey123	Некоректний URL	Null значення	Помилка підключення

Метод generateContent(keywords)

№	Вхідні дані	Клас еквівалентності (EP)	Граничне значення (BVA)	Очікуваний результат
1	["SEO", "Laptop"]	Коректний список	> 0 елементів	Контент згенеровано
2	[]	Порожній список	0 елементів	Повідомлення про помилку
3	null	Некоректний тип	Null значення	Помилка генерації
4	["A"]	Коротке ключове слово	1 символ	Слово ігнорується
5	["AB"]	Коротке ключове слово	2 символи	Слово ігнорується
6	["SEO"]	Коректне ключове слово	3 символи	Додається до контенту

Метод deployContent(content)

№	Вхідні дані	Клас еквівалентності (EP)	Граничне значення (BVA)	Очікуваний результат
1	Підключений сайт + валідний контент	Коректні дані	Нормальний випадок	Контент опубліковано
2	Непідключений сайт	Некоректний стан сайту	isConnected = false	Помилка публікації
3	Порожній bodyText	Некоректний контент	" "	Контент не публікується
4	null контент	Некоректний об'єкт	Null значення	Помилка
5	Контент з 1 символом	Мінімальний валідний текст	1 символ	Контент опубліковано

4. Вихідний код тестового набору з коментарями.

Administrator.test.js

```
const Administrator = require('../classes/Administrator');
const Client = require('../classes/Client');

describe('Тестування класу Administrator', () => {

    // Техніка: Позитивний, ЕР (Створення адміністратора)
    test('Успішне створення адміністратора', () => {

        // Arrange & Act
        const admin = new Administrator(
            1,
            'admin@mail.com',
            'admin123'
        );

        // Assert
        expect(admin.adminId).toBe(1);
        expect(admin.email).toBe('admin@mail.com');
    });

    // Техніка: Позитивний, ЕР (Перевірка методу manageUsers)
    test('Метод manageUsers коректно обробляє список користувачів', () => {

        // Arrange
        const admin = new Administrator(
            1,
            'admin@mail.com',
            'admin123'
        );

        const users = [
```

```

        new Client(1, 'user1@mail.com', '123', 'Pro'),
        new Client(2, 'user2@mail.com', '456', 'Basic')
    ];

    const consoleSpy = jest.spyOn(console,
'log').mockImplementation();

    // Act
    admin.manageUsers(users);

    // Assert
    expect(consoleSpy).toHaveBeenCalledWith('Список
користувачів:');
    expect(consoleSpy).toHaveBeenCalledWith('-
user1@mail.com');
    expect(consoleSpy).toHaveBeenCalledWith('-
user2@mail.com');

    consoleSpy.mockRestore();
});

// Техніка: Негативний, BVA (Порожній список користувачів)
test('Метод manageUsers працює навіть з порожнім списком',
() => {

    // Arrange
    const admin = new Administrator(
        1,
        'admin@mail.com',
        'admin123'
    );

    const consoleSpy = jest.spyOn(console,
'log').mockImplementation();

```

```

    // Act
    admin.manageUsers([]);

    // Assert
    expect(consoleSpy).toHaveBeenCalledWith('Список
користувачів:');

    consoleSpy.mockRestore();
  });
});

```

Client.test.js

```

const Client = require('../classes/Client');
const WordPressSite = require('../classes/WordPressSite');

describe('Тестування методу connectSite класу Client', () => {

  // Техніка: Позитивний, ЕР (Клас еквівалентності)
  test('Успішне підключення сайту з правильним URL та
ключем', () => {
    // Arrange (Підготовка)
    const client = new Client(1, 'eva@mail.com', 'pass123',
'Pro');

    const validUrl = 'https://site.com';
    const validApiKey = 'apikey123';

    // Act (Дія)
    const result = client.connectSite(validUrl,
validApiKey);

    // Assert (Перевірка)
    expect(result).toBe(true);
    expect(client.sites.length).toBe(1);
  });
});

```

```

});

// Техніка: Негативний, BVA (Граничне значення - порожній
рядок)
test('Помилка підключення, якщо URL порожній', () => {
    // Arrange (Підготовка)
    const client = new Client(1, 'eva@mail.com', 'pass',
'Pro');
    const emptyUrl = '';
    const validApiKey = 'apikey123';

    // Act (Дія)
    const result = client.connectSite(emptyUrl,
validApiKey);

    // Assert (Перевірка)
    expect(result).toBe(false);
});

// Техніка: Негативний, BVA (Граничне значення - 4 символи)
test('Помилка підключення, якщо API ключ має 4 символи', ()
=> {
    // Arrange (Підготовка)
    const client = new Client(1, 'eva@mail.com', 'pass',
'Pro');
    const validUrl = 'https://site.com';
    const shortApiKey = '1234';

    // Act (Дія)
    const result = client.connectSite(validUrl,
shortApiKey);

    // Assert (Перевірка)
    expect(result).toBe(false);

```

```
});  
});
```

Preset.test.js

```
const Preset = require('../classes/Preset');  
const SEOContent = require('../classes/SEOContent');  
  
describe('Тестування класу Preset', () => {  
  
    // 9. Техніка: Позитивний, ЕР (Створення об'єкта)  
    test('Успішне створення пресету з правильними параметрами',  
    () => {  
        // Arrange & Act  
        const preset = new Preset(101, 'Офіційний', 'UK');  
  
        // Assert  
        expect(preset.toneOfVoice).toBe('Офіційний');  
        expect(preset.language).toBe('UK');  
    });  
  
    // 10. Техніка: Позитивний, ЕР (Модифікація контенту)  
    test('Метод applyToContent коректно додає теги до тексту',  
    () => {  
        // Arrange  
        const preset = new Preset(101, 'Дружній', 'EN');  
        const content = new SEOContent(1, 'Смартфон');  
        content.bodyText = "Купуйте новий телефон.";  
  
        // Act  
        preset.applyToContent(content);  
  
        // Assert  
        expect(content.bodyText).toContain('[Мова: EN, Tone:  
Дружній]');
```

```
    });  
  });
```

SEOContent.test.js

```
const SEOContent = require('../classes/SEOContent');  
  
describe('Тестування методу generateContent класу SEOContent',  
  () => {  
  
    // Техніка: Позитивний, BVA (Слово рівно 3 символи - на  
    // межі, і воно проходить!)  
    test('Успішне додавання слова з мінімальною довжиною (3  
    символи)', () => {  
      // Arrange (Підготовка)  
      const seo = new SEOContent(103, 'Програмування');  
      const keywords = ['SEO', 'Mac']; // Рівно 3 символи  
      // (межа)  
  
      // Act (Дія)  
      seo.generateContent(keywords);  
  
      // Assert (Перевірка)  
      expect(seo.bodyText).toContain('SEO');  
      expect(seo.bodyText).toContain('Mac');  
    });  
  
    // Техніка: Негативний, EP (Передаємо зовсім неправильний  
    // тип даних, не масив)  
    test('Помилка генерації, якщо передано null замість  
    масиву', () => {  
      // Arrange (Підготовка)  
      const seo = new SEOContent(104, 'Помилка типу');
```



```

        const invalidData = null; // Негативний еквівалентний
клас (не масив)

        // Act (Дія)
        seo.generateContent(invalidData);

        // Assert (Перевірка)
        // Оскільки метод видає return при null і не змінює
текст, він має залишитися порожнім
        expect(seo.bodyText).toBe("");
    });

});

```

User.test.js

```

const User = require('../classes/User');
const Client = require('../classes/Client'); // Використовуємо
нащадка для тестування методів

describe('Тестування абстрактного класу User', () => {

    // 1. Техніка: Негативний, ЕР (Спроба ініціалізації
абстрактного класу)
    test('Помилка при спробі створити об'єкт класу User
безпосередньо', () => {
        // Arrange & Act & Assert
        expect(() => {
            new User('test@mail.com', 'pass123');
        }).toThrow("Неможливо створити об'єкт абстрактного
класу User");
    });

    // 2. Техніка: Позитивний, ЕР (Правильні дані для входу)

```

```
test('Успішний логін з правильним email та паролем', () =>
{
    // Arrange
    const client = new Client(1, 'test@mail.com',
'pass123', 'Pro');

    // Act
    const result = client.login('test@mail.com',
'pass123');

    // Assert
    expect(result).toBe(true);
});

// 3. Техніка: Негативний, ЕР (Неправильний пароль)
test('Помилка логіну, якщо введено неправильний пароль', ()
=> {
    // Arrange
    const client = new Client(1, 'test@mail.com',
'pass123', 'Pro');

    // Act
    const result = client.login('test@mail.com',
'wrong_pass');

    // Assert
    expect(result).toBe(false);
});

// 4. Техніка: Негативний, ЕР (Неправильний email)
test('Помилка логіну, якщо введено неправильний email', ()
=> {
    // Arrange
```

```
        const client = new Client(1, 'test@mail.com',
'pass123', 'Pro');

        // Act
        const result = client.login('wrong@mail.com',
'pass123');

        // Assert
        expect(result).toBe(false);
    });
});
```

WordPressSite.test.js

```
const WordPressSite = require('../classes/WordPressSite');
const SEOContent = require('../classes/SEOContent');

describe('Тестування методів класу WordPressSite', () => {

    // 5. Техніка: Позитивний, ЕР (Перевірка початкового стану)
    test('Новий сайт при створенні має статус "Не підключено"',
    () => {
        // Arrange
        const site = new WordPressSite(1,
'https://mysite.com');

        // Act
        const status = site.getStatus();

        // Assert
        expect(status).toBe("Не підключено");
        expect(site.isConnected).toBe(false);
    });
});
```

```
// 6. Техніка: Позитивний, ЕР (Зміна стану після виклику
методу)
test('Метод connect() змінює статус на "Підключено"', () =>
{
    // Arrange
    const site = new WordPressSite(1,
'https://mysite.com');

    // Act
    site.connect();
    const status = site.getStatus();

    // Assert
    expect(status).toBe("Підключено");
    expect(site.isConnected).toBe(true);
});

// 7. Техніка: Негативний, ЕР (Публікація без підключення)
test('Помилка deployContent, якщо сайт ще не підключений',
() => {
    // Arrange
    const site = new WordPressSite(1,
'https://mysite.com');
    const content = new SEOContent(1, 'Тест');
    content.bodyText = "Готовий текст";

    // Act
    const result = site.deployContent(content);

    // Assert
    expect(result).toBe(false);
});
```

```

// 8. Техніка: Негативний, BVA (Граничне значення: порожній
текст контенту)
test('Помилка deployContent, якщо текст контенту порожній',
() => {
    // Arrange
    const site = new WordPressSite(1,
'https://mysite.com');
    site.connect();
    const emptyContent = new SEOContent(1, 'Тест');
    emptyContent.bodyText = ""; // Межа - порожній рядок

    // Act
    const result = site.deployContent(emptyContent);

    // Assert
    expect(result).toBe(false);
});
});

```

5. Скріншот або HTML-звіт покриття коду з відображенням відсотку line coverage.

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Line #s
All files	94.2	91.66	93.33	94.2	
Administrator.js	100	100	100	100	
Client.js	100	100	100	100	
Preset.js	100	100	100	100	
SEOContent.js	100	83.33	100	100	17
User.js	90.9	100	66.66	90.9	25
WordPressSite.js	80	83.33	100	80	22-25

Test Suites: 6 passed, 6 total
 Tests: 18 passed, 18 total
 Snapshots: 0 total
 Time: 0.751 s, estimated 3 s
 Ran all test suites.

Do you want extension fro

6. Посилання на Git-репозиторій з кодом модуля та тестами.

<https://github.com/artemrohozian/OPI-project-ContentFlow>

b. Бонус: налаштувати автоматичний запуск тестів через GitHub Actions

test.yml

```
name: Node.js Tests
```

```
on:
```

```
  push:
```

```
    branches:
```

- main
- master

```
  pull_request:
```

```
    branches:
```

- main
- master

```
jobs:
```

```
  test:
```

```
    runs-on: ubuntu-latest
```

```
    strategy:
```

```
      matrix:
```

```
        node-version: [18.x]
```

```
    steps:
```

- name: Checkout repository
 uses: actions/checkout@v4
- name: Setup Node.js
 uses: actions/setup-node@v4

```

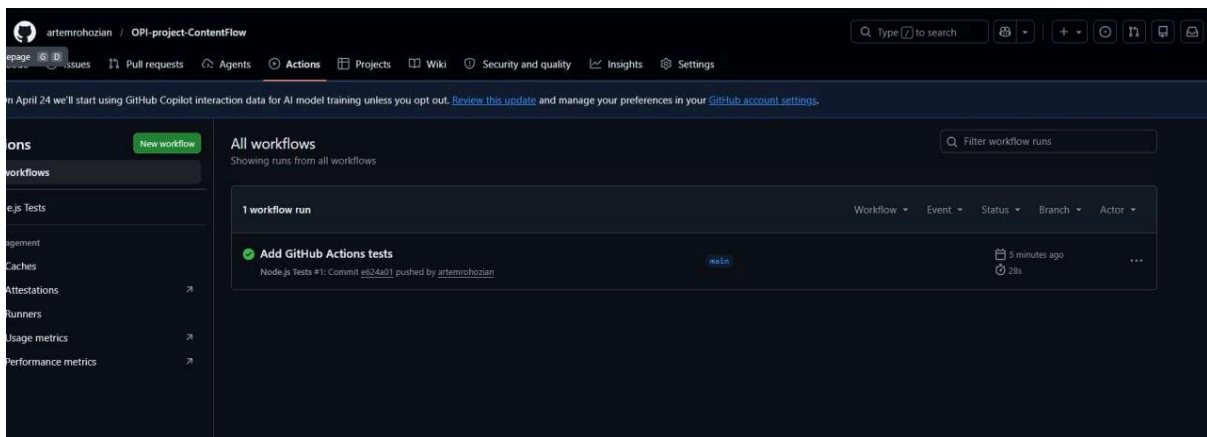
with:
  node-version: ${{ matrix.node-version }}

- name: Install dependencies
  run: npm install

- name: Run tests
  run: npm test

- name: Run coverage
  run: npm run test:coverage

```



7. Висновки (аналіз отриманих результатів покриття, виявлені проблеми, шляхи поліпшення).

Під час виконання лабораторної роботи було успішно застосовано методики проектування тестів. Для методів із нетривіальною логікою (`connectSite`, `generateContent`, `deployContent`) було виділено класи еквівалентності (EP) та визначено граничні значення (BVA).

На основі спроектованих таблиць було реалізовано набір модульних тестів мовою JavaScript із використанням фреймворку Jest. Усі тести суворо дотримуються патерну AAA (Arrange, Act, Assert). Перевірка покриття (Line Coverage) показала результат 94.2 %, що свідчить про високу якість тестового набору. Крім того, як бонус, було налаштовано автоматичний пайплайн у GitHub Actions для безперервної інтеграції (CI) та перевірки коду під час злиття гілок.